

חומרה מאיצה וכלכלת ה-AI

מדריך לימוד להרצאה: מהו CPU, למה הוא לא מספיק לעומסי AI, כיצד GPU משנה את התמונה, ואיך חומרה, זיכרון, חשמל ו-CUDA הפכו לחלק מכלכלת הבינה המלאכותית.

מוכן ללמידה + דף חזרה להרצאה של כ-10 דקות

מה המחשב באמת עושה, ומהו CPU?

כדי להבין למה עולם ה-AI זקוק למאיצים, צריך להתחיל מתפקידו של המעבד המרכזי. מחשב לא "מבין" טקסט, תמונה או קוד כפי שאדם מבין אותם. ברמת החומרה הכול מתורגם לרצפים של פעולות פשוטות: טעינת ערכים מהזיכרון, חיבור, כפל, השוואה, קפיצה לכתובת אחרת ושמירת תוצאה.

CPU – Central Processing Unit

ה-CPU הוא המעבד הכללי של המחשב. הוא חייב להתמודד היטב עם מגוון עצום של קוד: מערכת הפעלה, דפדפן, בסיס נתונים, תוכנת אבטחה, רשת, קבצים, פסיקות חומרה, הצפנה, חישובים וגם תנאים וקפיצות. בגלל זה הוא בנוי להיות **גמיש מאוד** ולהגיב מהר למשימות מורכבות ולא צפויות.

המטרה המרכזית

Low Latency

לסיים משימה בודדת ומורכבת במהירות גבוהה.

סגנון העבודה

מעט ליביות חזקות

כל ליבה כוללת הרבה לוגיקה לניהול קוד מורכב.

חוזקה

שליטה וקבלת החלטות

תנאים, הסתעפויות, קפיצות, גישה לזיכרון ותהליכים שונים.

מחיר

מורכבות בסיליקון

הרבה טרנזיסטורים מוקדשים לניהול, לא רק לכפל ולחיבור.

מה יש בתוך ליבת CPU?

ליבה מודרנית כוללת יחידות אריתמטיות, רגיסטרים, מטמוני L1/L2, מנגנוני טעינה ושמירה, מפענחי הוראות, מתזמנים, יחידות וקטוריות ומנגנונים מתקדמים שנועדו להסתיר עיכובים. שניים מהמנגנונים החשובים הם **Branch Prediction** ו-**Out-of-Order Execution**.

Branch Prediction: כאשר המעבד מגיע ל-if, הוא מנסה לנחש מראש באיזה מסלול הקוד ימשיך ומתחיל לעבוד לפני שהתשובה ודאית. אם ניחש נכון - נחסך זמן. אם טעה - הוא מבטל עבודה ומתחיל מחדש במסלול הנכון.

Out-of-Order: אם הוראה אחת ממתינה לנתון מהזיכרון, המעבד יכול לקדם הוראות אחרות שאינן תלויות בה. המטרה היא לא להשאיר את יחידות החישוב מובטלות.

ה-CPU אינו "מעבד חלש". להפך: הוא מעבד מתוחכם מאוד. הבעיה היא שחלק גדול מהתחכום שלו אינו מה שה-AI צריך בקנה מידה עצום.

הבעיה: AI לא מבקש "משימה אחת חכמה" - הוא מבקש המון מתמטיקה דומה

רשת נוירונים מבצעת שוב ושוב פעולות מספריות דומות על כמויות גדולות של נתונים. נוירון פשוט מקבל קלטים x , מכפיל כל קלט במשקל w , מחבר את התוצאות ומעביר אותן הלאה.

$$y = X_1W_1 + X_2W_2 + X_3W_3 + \dots$$

כאשר יש אלפי קלטים ואלפי נוירונים, כבר לא נוח לחשוב על כל נוירון בנפרד. מסדרים את הנתונים בתוך **וקטורים ומטריצות**. כך חישוב של שכבה שלמה מקבל צורה דומה ל:

$$Y = X \cdot W$$

X היא מטריצת הקלטים, W היא מטריצת המשקלים שנלמדו, ו- Y היא התוצאה. הכפלת מטריצות היא לא "פעולה אחת" אלא אוסף גדול מאוד של פעולות **Multiply-Accumulate**: כפל ועוד חיבור.

למה זה משנה את סוג החומרה שאנחנו רוצים?

בחישוב מטריצה גדולה קיימת מקביליות עצומה. הרבה תוצאות אינן תלויות זו בזו ולכן אפשר לחשב אותן באותו זמן. אם יש לנו מיליוני פעולות פשוטות ודומות, לא בהכרח כדאי להקצות לכל אחת ליבה מורכבת עם מנגנוני ניחוש, מטמונים גדולים ולוגיקת שליטה כבדה.

CPU mindset	AI workload
Few complex tasks	Huge number of similar operations
↓	↓
[Smart core] [Smart core]	x x x x x x x x x
[Smart core] [Smart core]	+ + + + + + + + +
↓	↓
Optimize latency	Optimize total throughput

Latency מול Throughput

Latency שואל: כמה מהר סיימנו משימה אחת? **Throughput** שואל: כמה עבודה כוללת הצלחנו להעביר בשנייה? CPU נבנה היסטורית עם דגש חזק על latency של זרמי קוד כלליים. AI בקנה מידה גדול דורש בעיקר throughput עצום על חישובים מספריים.

האנלוגיה: אם יש לך 10 משימות שונות ומסובכות, עדיף צוות קטן של מומחים. אם יש לך מיליון טפסים כמעט זהים שצריך לחשב עליהם את אותה נוסחה, עדיף אולם עם אלפי עובדים שעובדים במקביל.

אבל CPU כן מקבילי

חשוב לדייק: הטענה "CPU עובד בטור ו-GPU במקביל" אינה נכונה. CPU מודרני כולל כמה וכמה ליבות, SMT ויחידות וקטוריות כמו AVX. גם הוא מסוגל לבצע הרבה חישובים במקביל. ההבדל הוא **כמה סיליקון מוקדש למקביליות מספרית לעומת לוגיקת שליטה כללית**.

GPU: הרבה יחידות חישוב שמטרתן להעביר מסה של עבודה

ה-GPU נולד מעולם הגרפיקה. כדי לצייר תמונה מודרנית צריך לחשב צבע, תאורה, טקסטורה וגאומטריה עבור מספר עצום של פיקסלים וקדקודים. הרבה מהעבודה הזאת זהה בצורתה וניתנת לביצוע במקביל - ולכן נבנתה חומרה שמתאימה בדיוק לזה.

מאפיין	CPU	GPU
מטרה	ביצוע קוד כללי ומורכב	קצב עבודה מקבילי גבוה
יחידות חישוב	מעט יחסית, חזקות ומורכבות	כמות גדולה מאוד של יחידות פשוטות יותר
הסתעפויות	מצוין בתנאים וקוד לא צפוי	פחות יעיל כאשר threads מתפצלים למסלולים שונים
AI / מטריצות	מסוגל לבצע	מתאים מאוד בגלל מקביליות גבוהה
דגש	Latency	Throughput

איך GPU מארגן עבודה?

במונחים של NVIDIA, ה-GPU מחולק ליחידות בשם Streaming Multiprocessors (SMs). בכל SM נמצאות יחידות חישוב, רגיסטרים, זיכרון משותף ומתזמנים. תוכנית שמורצת על GPU יוצרת מספר גדול של threads, והחומרה מקבצת אותם לקבוצות ביצוע.

One operation, many data elements

```
Thread 0 → A[0] × B[0]
Thread 1 → A[1] × B[1]
Thread 2 → A[2] × B[2]
Thread 3 → A[3] × B[3]
...
```

Thousands of lightweight threads are scheduled across the GPU

Warps ו-SIMT

הגישה של NVIDIA נקראת SIMT - Single Instruction, Multiple Threads. קבוצת threads מבצעת אותה הוראה על נתונים שונים. ב-NVIDIA קבוצת ביצוע בסיסית נקראת warp. זה יעיל במיוחד כאשר כולם רוצים לבצע את אותו חישוב.

מה קורה עם תנאים?

אם חלק מה-threads בתוך אותה קבוצה רוצים לבצע צד אחד של if, וחלק רוצים את הצד השני, נוצרת **Branch Divergence**. החומרה עשויה לבצע את המסלולים בנפרד כאשר חלק מה-threads לא פעילים בכל מסלול. לכן GPU אוהב עבודה אחידה ומקבילית, ופחות אוהב זרמי קוד שכל אחד מהם מתנהג אחרת.

GPU אינו "CPU עם יותר ליבות". הוא ארכיטקטורה שבחרה להשקיע הרבה יותר מהטרניסטורים שלה בחישוב מקבילי וב-throughput.

למה כפל מטריצות הוא לב החיבור בין AI ל-GPU?

ניקח שתי מטריצות קטנות. כל תא בתוצאה נוצר מחיבור של מכפלות. במטריצות גדולות, מספר פעולות הכפל והחיבור גדל במהירות עצומה.

$$C[i,j] = \sum A[i,k] \cdot B[k,j]$$

לדוגמה, הכפלה של מטריצה בגודל 4096×4096 עם אוסף גדול של וקטורים יכולה כבר לדרוש מיליארדי פעולות. ברשת ניורונים לא מבצעים הכפלה אחת כזאת אלא שכבות רבות, שוב ושוב, על batches של נתונים.

Tensor Cores - התמחות נוספת בתוך ה-GPU

לאחר ש-GPU הפך לפלטפורמה מרכזית ל-Deep Learning, יצרניות החומרה הוסיפו יחידות ייעודיות שמאיצות את הפעולה הנפוצה ביותר בעולם הזה: **Matrix Multiply-Accumulate**. ב-NVIDIA הן נקראות Tensor Cores.

$$D = A \times B + C$$

במקום להסתמך רק על יחידות חישוב כלליות, Tensor Cores בנויים לבצע בלוקים של פעולות מטריצה ביעילות גבוהה מאוד. זה מגדיל את הביצועים ל-AI בלי להפוך את כל ה-GPU ל-ASIC קשיח לחלוטין.

Precision: למה AI לא תמיד צריך 32 או 64 ביט?

הרבה אלגוריתמי AI סובלים היטב דיוק מספרי נמוך יותר. לכן משתמשים בסוגי נתונים כמו FP8, BF16, FP16 וב-inference לעיתים גם INT8/INT4.

פחות זיכרון

מספר קטן יותר של ביטים לכל משקל מאפשר להחזיק מודל גדול יותר באותה קיבולת.

פחות תעבורה

כל קריאה מה-HBM מעבירה יותר ערכים באותו רוחב פס.

יותר חישוב

ניתן להכניס יותר יחידות חישוב ולבצע יותר פעולות בכל מחזור.

מחיר: דיוק

צריך לשמור על יציבות נומרית, ולכן באימון משתמשים לעיתים ב-Mixed Precision.

FLOPS אינם כל הסיפור

FLOP היא פעולת נקודה צפה. נתוני TFLOPS/PFLOPS מראים פוטנציאל חישובי, אבל ביצועים בפועל תלויים גם בזיכרון, בגודל ה-batch, בניצולת, בתקשורת בין מאיצים ובכמה האלגוריתם מצליח לשמור את יחידות החישוב עסוקות.

איך כל זה בא לידי ביטוי ב-LLM וב-Transformer?

מודל שפה גדול לא "קורא" מילים ישירות. הטקסט מפוצל ל-tokens, וכל token מיוצג כווקטור מספרי. מרגע זה, רוב העבודה המרכזית היא העברת מטריצות ו-tensors דרך שכבות מתמטיות.

שלב 1 - Embedding

כל token מקבל וקטור. אפשר לחשוב עליו כנקודה במרחב בעל אלפי ממדים. הווקטורים של כל הטוקנים יוצרים מטריצה X .

שלב 2 - Attention

ב-Transformer מחשבים שלושה ייצוגים מרכזיים - Q, K, V - באמצעות כפל של הקלט במטריצות משקלים:

$$Q = XW_q \quad K = XW_k \quad V = XW_v$$

אחר כך מחושב קשר בין tokens באמצעות פעולה שכוללת את QK^T , ולאחריה שוב כפל מטריצה עם V . כלומר גם ה-Attention עצמו נשען על Linear Algebra בקנה מידה גדול.

שלב 3 - Feed Forward

כל בלוק Transformer כולל גם רשת Feed-Forward. בצורה מופשטת:

$$H = \text{activation}(XW_1)W_2$$

שוב שתי הכפלות מטריצה גדולות. כאשר מכפילים את זה בעשרות או מאות שכבות, במספר גדול של tokens וב-batch גדול, קל להבין מאיפה מגיע הצורך במאיצים.

Training מול Inference

	Inference	Training	
מה עושים?	משתמשים במשקלים שכבר נלמדו	מלמדים את המודל ומשנים את המשקלים	
חישוב	Forward בלבד, token אחרי token	Forward + Loss + Backpropagation	
זיכרון	Weights + KV Cache + Activations	+ Weights + Activations + Gradients Optimizer	
אופי צוואר הבקבוק	לעיתים קרובות memory bandwidth, במיוחד ב-batch קטן	לעיתים compute ותקשורת	

נקודה חשובה להרצאה: ב-LLM, חומרה מאיצה אינה רק "מחשבת מהר יותר". היא מתוכננת בדיוק לצורה המתמטית שבה המודל מיוצג - tensors, מטריצות ופעולות מקביליות.

ה-GPU יכול להיות מהיר מאוד - ועדיין לחכות לזיכרון

כוח חישובי לבדו אינו מספיק. יחידת חישוב חייבת לקבל את הנתונים שעליהם היא עובדת. אם הנתונים מגיעים לאט יותר מהקצב שבו ה-GPU יכול לחשב, המאיץ יושב חלק מהזמן ללא עבודה. לכן בעולם AI מדברים ללא הפסקה על **Memory Bandwidth**.

HBM - High Bandwidth Memory

מאיצי AI מתקדמים משתמשים בזיכרון HBM שנמצא קרוב מאוד ל-GPU ומספק רוחב פס גבוה בהרבה מזיכרון מערכת רגיל. קיבולת HBM קובעת בין היתר כמה מהמודל ניתן להחזיק על המאיץ אחד; רוחב הפס קובע כמה מהר ניתן להזרים את המשקלים וה-activations ליחידות החישוב.

דוגמה: מודל עם 70 מיליארד parameters, כאשר כל parameter נשמר ב-16 ביט, דורש בערך 140GB רק למשקלים. באימון נדרשים בנוסף gradients, optimizer states ו-activations - ולכן הצריכה יכולה להיות גבוהה בהרבה.

למה אי אפשר פשוט לשים הכול ב-RAM?

RAM רגיל יכול להיות גדול וזול יותר, אבל המרחק ורוחב הפס שלו אינם מתאימים לקצב שה-GPU דורש. העתקת נתונים בין CPU memory ל-GPU memory היא גם פעולה יקרה. לכן frameworks מנסים לשמור נתונים קריטיים קרוב למאיץ ולמחזר אותם ככל האפשר.

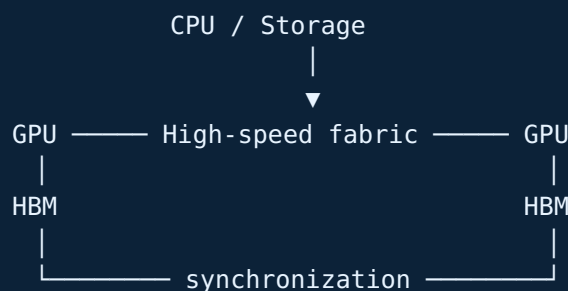
כאשר GPU אחד לא מספיק

מודלים גדולים ואימונים גדולים מתפרסים על הרבה GPUs. קיימות כמה אסטרטגיות:

- **Data Parallelism:** GPUs שונים מעבדים batches שונים ומסנכרנים gradients.
- **Tensor/Model Parallelism:** מפצלים מטריצות או שכבות של אותו מודל בין מאיצים.
- **Pipeline Parallelism:** קבוצות GPUs מבצעות חלקים שונים של רצף השכבות.

ואז הרשת הופכת לחלק מהמחשב

אם אלפי GPUs חייבים לסנכרן מידע בכל צעד אימון, התקשורת ביניהם יכולה לקבוע את ביצועי המערכת כולה. לכן טכנולוגיות כמו NVLink, NVSwitch, Ethernet ו-InfiniBand מהיר הן חלק אינטגרלי מתשתית AI. ברגע שעוברים ל-cluster, "המחשב" כבר אינו שבב - הוא מערכת שלמה.



Performance = Compute + Memory + Interconnect + Software

למה ענקיות הענן מפתחות שבבי AI משלהן?

אפשר לחשוב על מעבדים כספקטרום בין גמישות ליעילות. CPU נמצא בצד הגמיש ביותר; GPU מקריב חלק מהגמישות כדי לקבל מקביליות גבוהה; ו-ASIC נבנה מראש עבור קבוצת משימות מצומצמת יותר ולכן יכול להיות יעיל מאוד.

סוג	יתרון	חסרון	דוגמאות
CPU	גמישות מקסימלית	פחות יעיל למטריצות ענק	Intel Xeon, AMD EPYC
GPU	מקביליות גבוהה + גמישות	צריכת חשמל ועלות גבוהות	NVIDIA, AMD Instinct
AI ASIC	יעילות גבוהה ל-workload מסוים	פחות כללי ותלוי ecosystem	Google TPU, AWS Trainium

Google TPU

TPU - Tensor Processing Unit הוא מאיץ ש-Google פיתחה סביב עומסי machine learning. חלק מרכזי בגישה הוא יחידות מטריצה ייעודיות, לעיתים במבנה systolic array, שבהן נתונים זורמים בין יחידות Multiply-Accumulate סמוכות. כך אפשר לבצע reuse לנתונים ולהקטין גישות יקרות לזיכרון.

AWS Trainium

Amazon מפתחת את משפחת Trainium עבור workloads של AI בענן שלה, עם שכבת תוכנה בשם AWS Neuron. הרעיון העסקי חשוב לא פחות מהטכני: אם AWS יכולה לבצע חלק מה-AI של לקוחותיה על חומרה שהיא מתכננת, היא מפחיתה תלות בספק חיצוני ויכולה לבצע אופטימיזציה של עלות לכל workload.

AMD

AMD היא המתחרה המרכזית של NVIDIA בעולם המאיצים הכלליים עם משפחת Instinct ו-ecosystem בשם ROCm. מבחינה טכנית תחרות חומרה בלבד אינה מספיקה - היא חייבת להגיע יחד עם compilers, kernels, libraries, framework support וכלי תפעול.

חברת ענן אינה שואלת רק "איזה שבב הכי מהיר?" אלא "כמה עולה לי לייצר מיליון tokens, ומה מידת השליטה שלי בכל ה-stack?"

למה היתרון של NVIDIA הוא הרבה יותר מסיליקון?

כאשר אומרים "NVIDIA שולטת בשוק ה-AI", קל לחשוב שהסיבה היא רק GPU מהיר. בפועל, היתרון הגדול שלה הוא **מערכת שלמה**: חומרה, תקשורת, ספריות, compiler, כלים ומיליוני מפתחים שכבר יודעים לעבוד איתה.

CUDA כ-ecosystem

CUDA היא פלטפורמת החישוב וה-programming model של NVIDIA ל-GPU. מסביבה נבנו שכבות כמו:

cuBLAS

Linear Algebra ופעולות מטריצה מותאמות.

cuDNN

primitives של Deep Learning.

NCCL

תקשורת יעילה בין GPUs ב-training מבוזר.

TensorRT

אופטימיזציה ו-runtime ל-inference.

מעל הספריות האלה נמצאים frameworks של PyTorch, LLM, כלים ל-debugging, profiling, serving, לכן ארגון שעובד שנים עם NVIDIA אינו מחזיק רק כרטיסים פיזיים; יש לו קוד, ידע, תהליכים, kernels, monitoring ונהלי תפעול שנבנו סביב הפלטפורמה.

איך נוצר Lock-in?

1 יותר חברות רוכשות NVIDIA כי התמיכה בכלים וב-frameworks טובה.

2 יותר מפתחים כותבים ומבצעים אופטימיזציה ל-CUDA.

3 יותר ספריות וכלי תוכנה עובדים הכי טוב או נבדקים קודם על NVIDIA.

4 המעבר לספק אחר דורש לא רק החלפת שבב אלא התאמות software ו-operations.

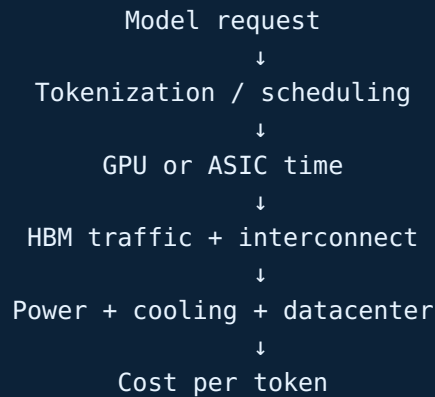
5 העלות של המעבר מחזקת את הבחירה הקיימת ומגדילה את אפקט הרשת.

האם זה מונופול?

מבחינה מקצועית עדיף לתאר זאת כ**עמדה דומיננטית עם moat תשתיתי ו-software lock-in**, ולא לקבוע אוטומטית "מונופול" במובן משפטי. קיימות אלטרנטיבות - AMD, TPUs, Trainium ומאיצים נוספים - אבל מתחרה חייבת להתחרות ב-stack שלם, לא רק בנתון FLOPS.

איך כוח מחשוב הפך למשאב כלכלי?

כל תשובת AI צורכת זמן מאיץ, זיכרון, רשת, חשמל וקירור. כאשר שירות משרת מיליוני משתמשים, ההפרש בין ארכיטקטורה יעילה ולא יעילה יכול להיות עצום. מכאן מגיע מושג מרכזי: **Cost per Token** - העלות להפיק יחידת פלט.



למה hyperscalers רוצים שבבים משלהם?

Google, AWS וחברות ענן אחרות מפעילות workload עצום וצפוי יחסית. אם הן יכולות לתכנן חומרה, compiler ו-data center יחד, הן יכולות לשפר **performance per watt**, להקטין עלות ולצמצם תלות ב-NVIDIA. במקביל הן עדיין יכולות להציע GPUs ללקוחות שזקוקים לגמישות ול-CUDA.

AI הוא כבר לא עסק של תוכנה בלבד

כדי להפעיל מודלים בקנה מידה גדול נדרשת שרשרת תעשייתית שלמה:

- תכנון שבבים ומאיצים.
- מפעלי ייצור מתקדמים ו-lithography.
- Advanced packaging וחיבור HBM-chiplets.
- זיכרון מהיר, storage-I networking.
- מרכזי נתונים, קירור ותשתית חשמל.
- שכבת תוכנה שמצליחה לנצל את כל החומרה.

המשמעות: היכולת לפתח AI מתקדם תלויה לא רק באלגוריתם וב-data, אלא גם בגישה ל-compute. לכן GPUs, fabs, HBM, power ו-data centers הפכו לנכסים אסטרטגיים.

למה היעילות חשובה כל כך?

אם אופטימיזציה מורידה את זמן ה-inference ב-20%, היא לא רק "עושה את המודל מהיר יותר" - היא עשויה לאפשר לאותו cluster לשרת יותר משתמשים, לדחות רכישת חומרה נוספת ולהקטין צריכת חשמל. בעולם שבו המאיצים יקרים והביקוש גבוה, ביצועי תוכנה מתורגמים ישירות לכסף.

האם צוואר הבקבוק הבא הוא חומרתי או אלגוריתמי?

התשובה הטובה היא: שניהם. ככל שהמודלים גדלים, צוואר הבקבוק זז בין compute, זיכרון, תקשורת, ייצור, חשמל וקירור. במקביל, שיפור אלגוריתמי יכול להקטין דרמטית את כמות המשאבים הדרושה.

ייצור סיליקון

שבבים מתקדמים דורשים מפעלי ייצור יקרים וקיבולת מוגבלת.

Lithography

תהליכים מתקדמים תלויים בצידוד ייצור מורכב ביותר ובשרשרת אספקה מצומצמת.

Advanced Packaging

compute dies, HBM משלבים AI accelerators וחיבורים מהירים בצפיפות גבוהה.

HBM

גם שבב חזק אינו שימושי אם אין מספיק זיכרון מהיר וקיבולת מתאימה.

Networking

ב-clusters גדולים, זמן synchronization עלול להפוך לחלק משמעותי מהאימון.

חשמל וקירור

Data center צריך לקבל הספק עצום ולהוציא את אותו סדר גודל של חום.

ומה הצד האלגוריתמי עושה?

- **Quantization**: שימוש ב-FP8/INT8/INT4 כדי להקטין זיכרון וחישוב.
- **Sparsity**: הימנעות מחישוב על ערכים שאינם תורמים.
- **Mixture of Experts**: הפעלת רק חלק מהמומחים לכל token במקום כל פרמטר במודל.
- **Distillation**: אימון מודל קטן לחקות מודל גדול.
- **Kernel/Memory optimization**: אלגוריתמים כמו attention יעיל יותר מצמצמים תנועת נתונים.

הפרדוקס של יעילות

אם העלות ל-token יורדת, השירות נעשה זול ונגיש יותר - ואז משתמשים בו יותר. לכן שיפור יעילות לא בהכרח מוריד את הצריכה הכוללת; הוא יכול להגדיל את הביקוש. זה דומה ל-Jevons Paradox: משאב יעיל וזול יותר נעשה שימושי ליותר מקרים.

האתגר הבא של AI אינו "לבנות GPU מהיר יותר" בלבד. הוא לבנות מערכת שבה software-*i* compute, memory, network, power, cooling מתקדמים יחד.

מסלול מוכן להרצאה של כ-10 דקות

1 דקה 0-1: פתיחה. "AI נראה כמו תוכנה, אבל מתחתיו קיימת תעשיית חומרה אדירה. כדי להבין למה NVIDIA הפכה כל כך מרכזית, צריך להבין קודם למה CPU לבדו אינו מתאים לסוג החישוב הזה."

2 דקה 1-3: CPU. הסבר שהוא general purpose, בנוי ל-latency, תנאים וקוד מורכב, עם מעט ליבות חזקות, out-of-order, cache, branch prediction.

3 דקה 3-4: הבעיה של AI. הצג את הנוסחה $Y=XW$. הסבר שרשת נוירונים היא במידה רבה מיליארדי פעולות Multiply-Accumulate על מטריצות.

4 דקה 4-6: GPU. הסבר throughput, אלפי SIMT, threads, ולמה אותה פעולה על הרבה נתונים מתאימה ל-GPU. הזכר Tensor Cores.

5 דקה 6-7: LLM. Feed Forward - Q, K, V, Attention - כולם חוזרים שוב ושוב לכפל מטריצות.

6 דקה 7-8: שוק. NVIDIA/AMD כ-GPUs כלליים, Google TPU ו-AWS Trainium כ-ASICs ממוקדים.

7 דקה 8-9: CUDA. NVIDIA אינה רק שבב; היא ecosystem. המעבר לספק אחר כולל software, libraries, networking וידע.

8 דקה 9-10: עתיד. הסבר שהחסמים הם electricity, networking, packaging, silicon, HBM ו-cooling - מול אלגוריתמים שמנסים לעשות יותר עם פחות.

שלושה משפטים שכדאי לזכור בעל פה

1. CPU מותאם לסיים כמה משימות מורכבות מהר; GPU מותאם להעביר כמות עצומה של חישובים דומים במקביל.

2. החיבור בין GPU ל-AI נובע מהמתמטיקה: רשתות נוירונים מבצעות שוב ושוב כפל מטריצות, ולכן אפשר לנצל מקביליות עצומה.

3. היתרון של NVIDIA אינו רק GPU מהיר; CUDA וכל ה-ecosystem מסביב יוצרים lock-in וחוסם מעבר גבוה.

טיפ: אל תנסה להסביר כל פרט. המטרה היא שהקהל יבין את השרשרת: CPU → מגבלת GPU → throughput → מטריצות → AI → ecosystem → כלכלה ותשתיות.

שאלות שסביר שישאלו אותך

האם GPU תמיד מהיר יותר מ-CPU?

לא. GPU מנצח בעיקר כאשר קיימת מקביליות גבוהה ועבודה מספרית אחידה. קוד עם הרבה תנאים, I/O ושליטה מורכבת יכול להיות מתאים יותר ל-CPU.

למה לא להריץ את כל מערכת ההפעלה על GPU?

כי מערכת ההפעלה דורשת latency נמוך, branching, interrupts, memory management והרבה זרמי עבודה שונים - בדיוק המקום שבו CPU כללי יעיל וגמיש יותר.

האם AI חייב NVIDIA?

לא. הוא צריך מאיץ מתאים. אפשר להשתמש ב-AMD, TPU, Trainium ו-maizim נוספים. NVIDIA דומיננטית בגלל שילוב של חומרה ו-software ecosystem.

אם ASIC יעיל יותר, למה כולם לא עוברים ל-ASIC?

כי specialization בא על חשבון גמישות. מודלים ו-workloads משתנים מהר, ו-GPU מאפשר להריץ מגוון גדול של kernels ומודלים. ASIC משתלם במיוחד לחברה שמחזיקה workload עצום ויציב יחסית.

מה בעצם עושה Tensor Core?

הוא מאיץ בלוקים של פעולות matrix multiply-accumulate, שהן הלב של שכבות רבות ברשת נוירונים.

למה HBM כל כך חשוב?

כי יחידות החישוב צריכות לקבל weights ו-activations במהירות. אם רוחב הפס לזיכרון נמוך, ה-GPU ממתיין וה-FLOPS התיאורטיים לא מנוצלים.

מה ההבדל בין Training ל-Inference?

Training משנה את המשקלים באמצעות forward ו-inference; backpropagation משתמש במשקלים שכבר נלמדו כדי ליצור תחזית או token חדש.

מה צוואר הבקבוק הבא?

אין אחד. בחלק מהמערכות זה compute, באחרות power, network, HBM או cooling. ברמת התעשייה קיימים גם חסמי silicon manufacturing ו-advanced packaging.

האם אלגוריתם טוב יכול להחליף חומרה?

לא לחלוטין, אבל אופטימיזציה כמו quantization, sparsity או attention יעיל יותר יכולה לחסוך כמות משמעותית של compute וזיכרון - ולעיתים לתת אפקט ששווה לשדרוג חומרה.

משפט סיום

בעידן ה-AI, כוח מחשוב הוא כבר לא רק רכיב טכני מאחורי התוכנה. הוא משאב כלכלי ותשתיתי - שמחבר בין אלגוריתמים, שבבים, מפעלי ייצור, מרכזי נתונים, חשמל ושרשרת אספקה עולמית.